

Triage Platform — Load Test 3 (16 workers, large-file soak)

2026-06-30 · 16 worker_py containers @ 4 GB · PostgreSQL tuned (shared_buffers 4 GB, max_wal_size 8 GB) on the new **fastpool** (16K-recordsize SSD mirror) · 2-hour run

CLAIMS PROCESSED
13,443,911

FILES
7,617

DURATION
120 min

SUSTAINED
10,210/s

PEAK
48,330/s

Result

- **13.4 million claims processed in 2 hours on 16 workers** — sustained ~10,210 claims/s, peaking ~48,330/s on dense files. 100% success on the tiers that ran.
- The Postgres + storage work (test-2 follow-up) is the enabler: the DB now lives on a dedicated tuned SSD mirror, so 16 workers scale without the write contention that capped earlier runs.

Scaling progression (3 tests)

Test	Workers	Claims (2 h*)	Throughput	Notes
1	4	1.06 M (28 min)	~622 /s avg	baseline, mixed sizes
2	8	6.10 M	~910 /s loop avg; ~5,700 /s dense	pre-Postgres-tuning
3	16	13.44 M	~10,210 /s sustained; 48,330 peak	post Postgres+fastpool tuning

*Tests 2 & 3 are 2-hour soaks; test 1 is a single ~28-min pass. Corpora differ (test 3 is large-file-weighted), so treat as a directional progression, not a controlled benchmark.

Per-tier processed (this run)

Tier	Files processed	Note
50 MB	80	dense, drove peak throughput
10 MB	100	full tier
512 KB	400	full tier
1 KB	7,037	small-file tail (per-file-overhead bound)

Capacity finding — large single files. 10 MB and 50 MB files process cleanly at high throughput. **100 MB and 200 MB single files hit an architectural ceiling:** each file is base64-encoded into one RabbitMQ message (≈140 / 279 MB) and parsed entirely in worker memory (~3–6 GB for 250–500K claims), which overwhelms the broker and the per-worker memory budget regardless of worker count.

Recommendation: add chunked / streaming ingest (split very large flat files at the interchange boundary before enqueue) for files > ~50–70 MB — this is the next throughput unlock.

Live dashboard: <https://triage-report.192.168.1.206.nip.io/>